Steganography Tool for Image/File Hiding

1. Introduction

Steganography is the technique of hiding secret information inside digital media such as images, audio, or video files. Unlike encryption, which hides the content of a message, steganography hides the existence of the message itself. This project implements image-based steganography using the Least Significant Bit (LSB) technique. By modifying the least significant bits of pixel values in PNG or BMP images, secret text data can be embedded without causing noticeable changes to the original image. This ensures secure and covert communication.

2. Abstract

The Steganography Tool for Image/File Hiding is a Python-based application developed to securely embed and extract hidden messages within image files. The system uses the Pillow (PIL) library along with LSB steganography techniques to manipulate pixel data and store secret information. The tool provides a command-line interface (CLI) that allows users to input an image and a secret message for embedding, and later extract the hidden message from the encoded image. The project demonstrates a practical implementation of data hiding techniques and emphasizes the importance of secure communication in cybersecurity applications.

3. Tools Used

- Python 3.x
- Pillow (PIL) Library for image processing
- Stepic Library for LSB steganography
- Tkinter (Optional for GUI implementation)
- Command Line Interface (CLI) / IDLE

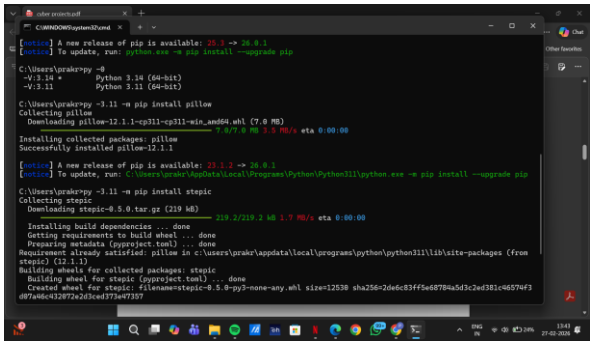
4. Steps Involved in Building the Project

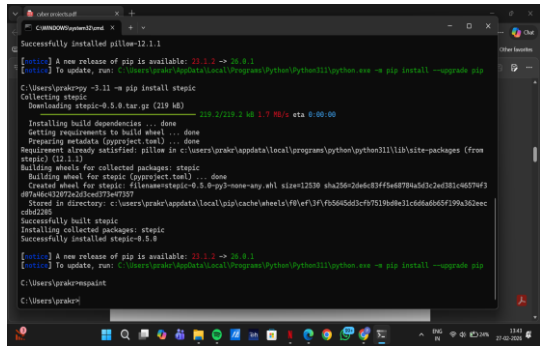
The project began with setting up the environment by installing Python and the required libraries (Pillow and Stepic) using pip. A dedicated project folder and main Python script file were created. Next, image handling functionality was implemented using PIL to open and process PNG/BMP image files while ensuring proper file path handling.

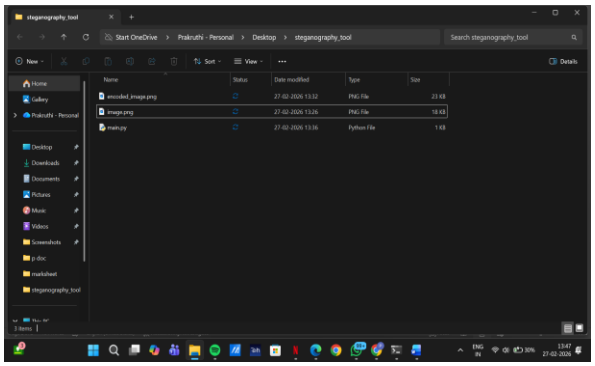
The secret message was converted into binary format and embedded into the least significant bits of image pixels using the LSB technique. A new encoded image (encoded_image.png) was generated without visibly altering the original image. A decoding function was then implemented to extract the hidden message from the modified image. Finally, testing and debugging were performed to resolve issues such as file path errors and decoding mismatches, ensuring smooth and reliable execution.

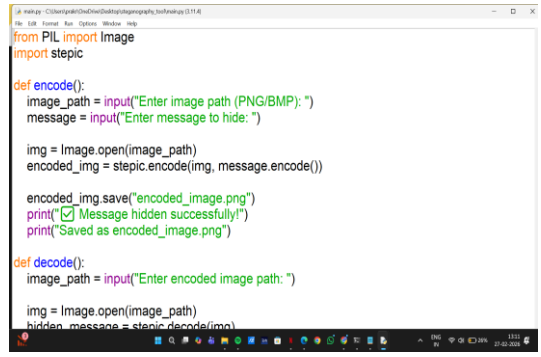
5. Conclusion

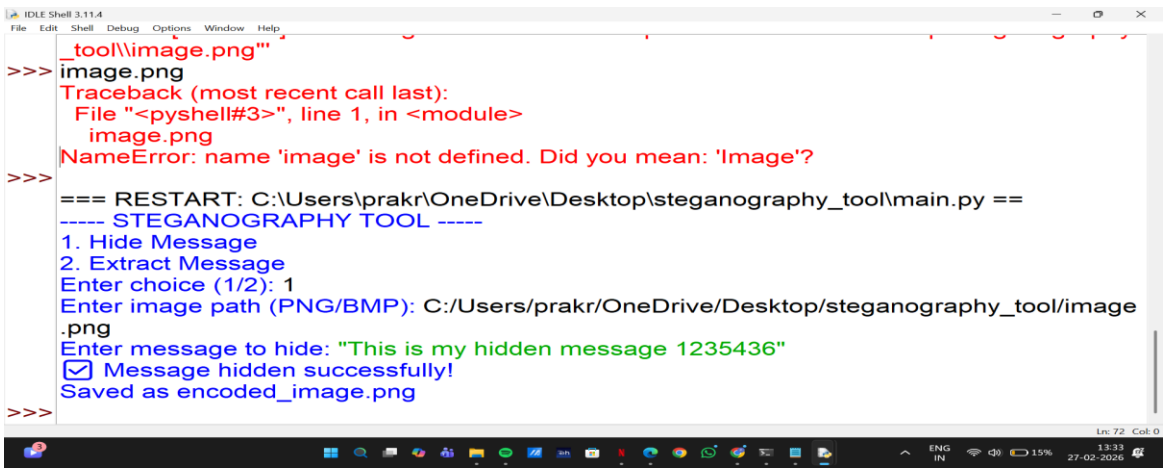
The Steganography Tool successfully demonstrates how secret messages can be securely embedded within images using LSB techniques. By implementing both embedding and extraction functionalities, the system provides a complete steganography solution. This project enhances understanding of image processing, data hiding methods, and practical cybersecurity concepts, making it a valuable academic and real-world security application.












```
>>>
=== RESTART: C:\Users\prakr\OneDrive\Desktop\steganography_tool\main.py ==
----- STEGANOGRAPHY TOOL -----
1. Hide Message
2. Extract Message
Enter choice (1/2): 2
Enter encoded image path: encoded_image.png
🔓 Hidden message is: "This is my hidden message 1235436"
>>>
```